Deep Reinforcement Learning

Model-based reinforcement learning

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

SciML

Content

Content

- Revisiting the past: Dyna-style algorithms
  - Dyna-Q and Dyna-$Q+$
  - Prioritized sweeping
- Looking into the future: Planning at decision time
  - Heuristic search
  - Rollouts
  - MCTS
- AlphaGo & AlphaZero
- Learning a model
  - Dreamer architecture

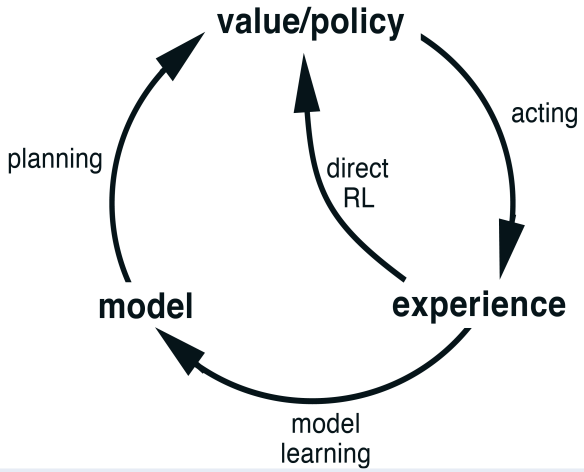
Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6-13	DQN, policy gradients, actor-critic, PPO/SAC, exploration	Deep RL from basics to modern algorithms
	Model-Based Control	
14	Optimal control & feedback control	How to do control when we know the model
15	Model-based reinforcement learning	RL with known and learned models
	Advanced Topics	

Lecture contents

Learning vs. planning in the RL context

- *Learning* and *planning* are closely related in RL (Sutton and Barto 1998).
- The key difference is the *origin of the training data*:
  - **Learning**: **real experience**,
  - **Planning**: **simulated experience**.
- Simplest example: Q-learning vs. Q-planning.



Algorithm: Q-learning
for $t = 0, 1, 2, \ldots$
Sample action $a_t \sim \pi(s_t)$
Observe (r_t, s_{t+1}) from **environment**
Update Q given (s_t, a_t, r_t, s_{t+1}):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)

Algorithm: Q-planning
for $t = 0, 1, 2, \ldots$
Sample action $a_t \sim \pi(s_t)$
Use **model** to create sample (r_t, s_{t+1})
Update Q given (s_t, a_t, r_t, s_{t+1}):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)

Unified view: create experience using a model $\Rightarrow$ model $\longrightarrow$ simulated experience $\xrightarrow{\text{backups}}$ values $\longrightarrow$ policy

Revisiting the past: Dyna algorithms

The Dyna-Q concept

Easiest way to create a model: Remember past a, $s \to s'$ and r.

Algorithm: Dyna-Q (Tabular RL)

loop forever:

Sample $a \sim \pi(\cdot|s, Q)$ (e.g. ϵ-greedy)

Take action a, observe reward r and next state s'

$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$

Add transition to our **model** of previous experiences

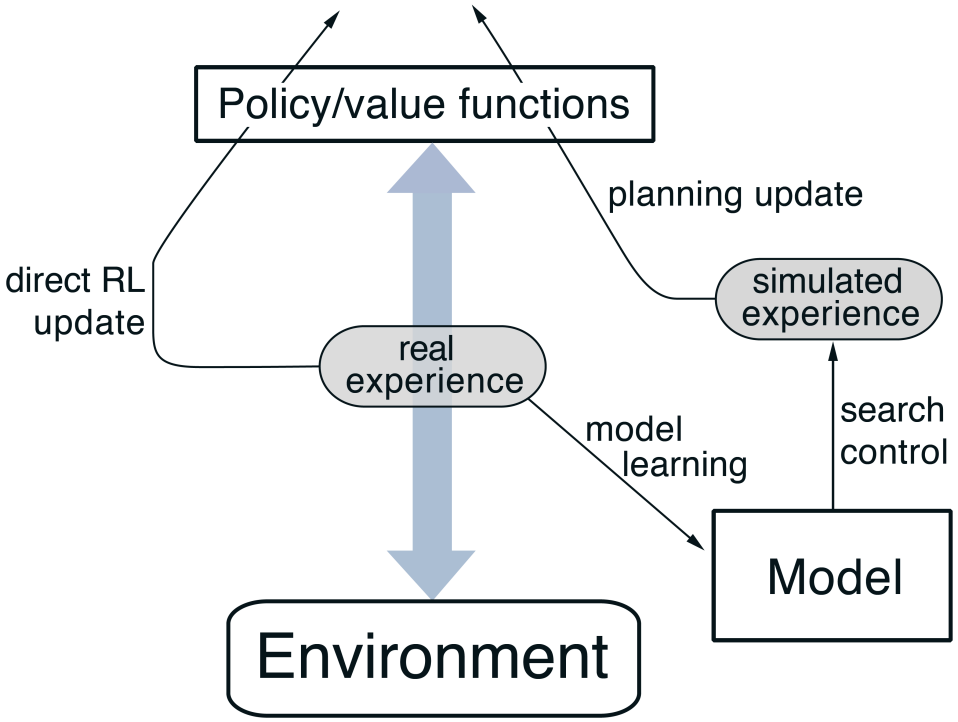
for $i = 1, \ldots, n$:

$s \leftarrow$ random previous state

$a \leftarrow$ random previous action

$r, s' \leftarrow \textbf{model}(s,a)$

$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$



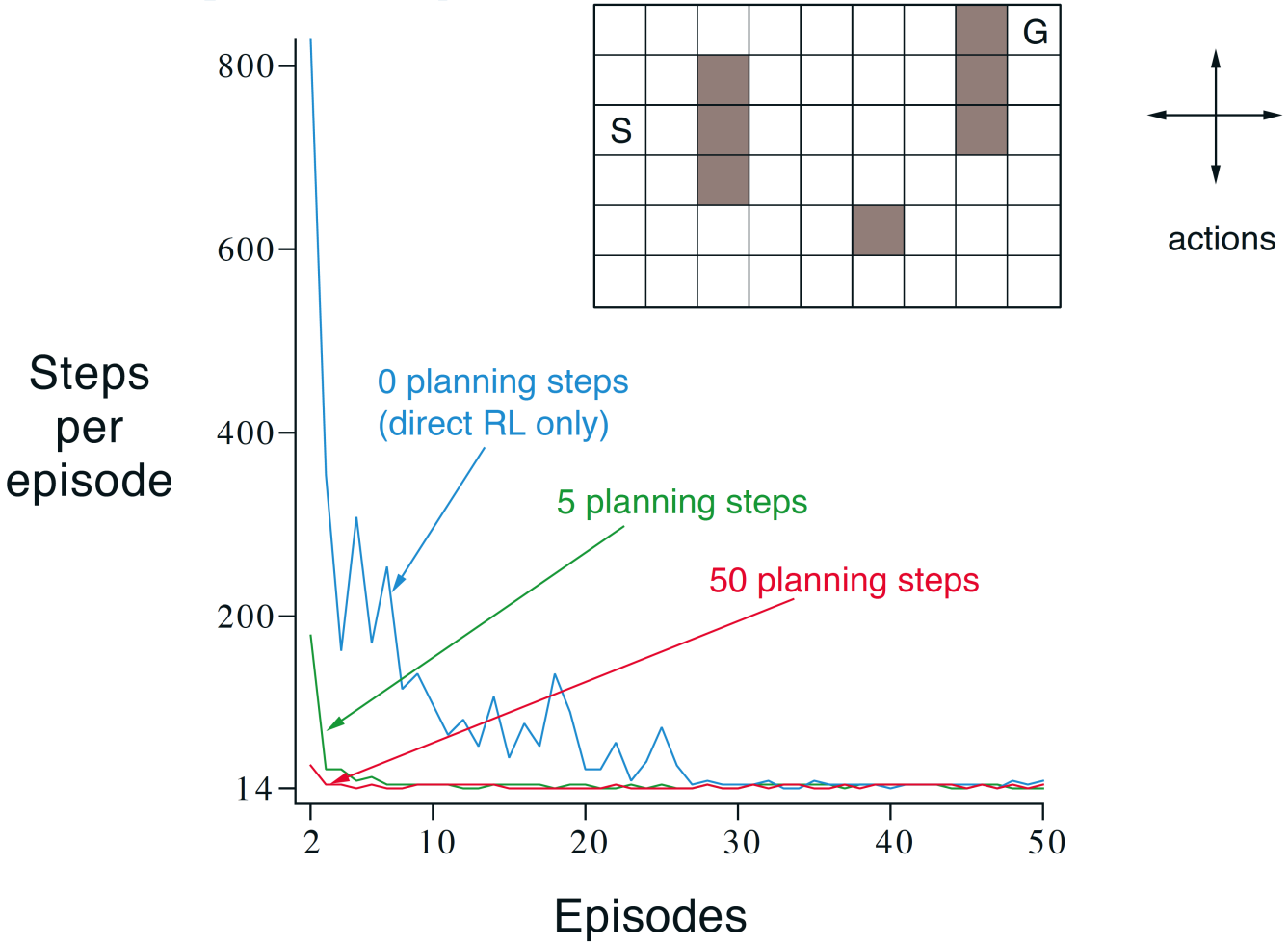
The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment (Source: (Sutton and Barto 1998, Fig. 8.1)).

Why does this even make sense? Aren't we just doing the exact same calculation?

$\Rightarrow$ our approximation of $Q(s,a)$ may be better later on!

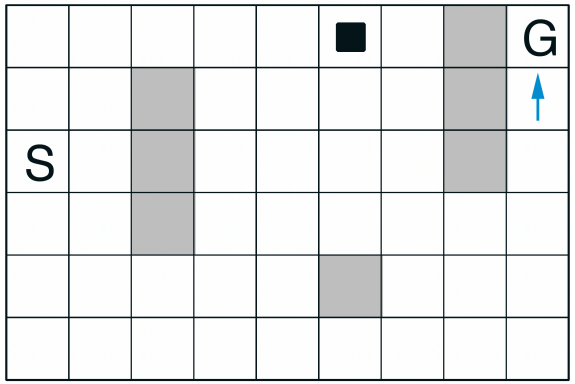
$\Rightarrow$ sparse rewards get "time to travel" through the environment!

Example: Dyna Maze (1)

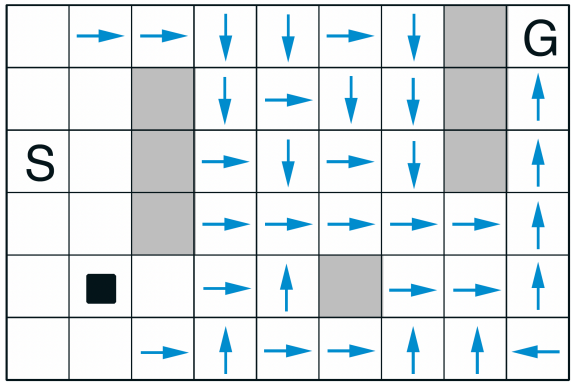


A simple maze (inset) and the average learning curves for Dyna-Q agents varying in their number of planning steps (n) per real step. The task is to travel from S to G as quickly as possible (Source: (Sutton and Barto 1998, Fig. 8.2)).

WITHOUT PLANNING (n=0)



WITH PLANNING (n=50)



Policies found by planning and nonplanning Dyna-Q agents halfway through the second episode. The arrows indicate the greedy action in each state; if no arrow is shown for a state, then all of its action values were equal. The black square indicates the location of the agent (Source: (Sutton and Barto 1998, Fig. 8.3)).

When the model is wrong

In the previous maze example, the environment was perfect and the transitions deterministic.

Reasons for incorrect or inaccurate memories:

- The environment is stochastic and only a **limited number of samples** have been observed.
- The model was learned using function approximation and has **generalized imperfectly**.
- The **environment has changed** and its new behavior has not yet been observed.

Can we recover from imperfect memories and adapt?

\+ If the old model is optimistic, there is a good chance!

— If the old model is pessimistic, it is very hard to find new opportunities!

- even with ϵ-greedy, it is very unlikely to randomly sample a sequence that ends up finding the better solution.
- the old **exploration-exploitation dilemma**!

Dyna-$Q+$

Two changes can help our simple Dyna-Q algorithm to foster exploration.

1. For each state, *count* the time τ since it has been revisited last (where $\kappa > 0$ is a hyperparameter) and augment the reward by a bonus:

$$\hat{r} = r + \kappa\sqrt{\tau}.$$

- The longer we have not revisited a state-action pair s, a, the larger the (boosted) reward for revisiting.

2. In the planning phase, we are allowed to try out new actions, even if we have not used them before.

- Since we cannot know the next state s' or the reward r in such a case, the initial setting is $s' = s$ and $r = 0$.
- Nevertheless, not trying out new actions results in them getting large τ and thus, large augmented rewards! $\Rightarrow$ Update towards higher Q estimate.
- Eventually, the learning phase will actually explore the new action.
  - If it truly is better: $\tau \leftarrow 0$, but confirmation by large r.
  - If it was optimistic: $\tau \leftarrow 0$, and rejection due to small r.

Algorithm: Dyna-Q (Tabular RL)

loop forever:

(*Learning / Experience*)

Sample action a ϵ-greedily

Take action a, observe r and s'

$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$

Add transition to our **model**

(*Planning*)

for $i = 1, \ldots, n$:

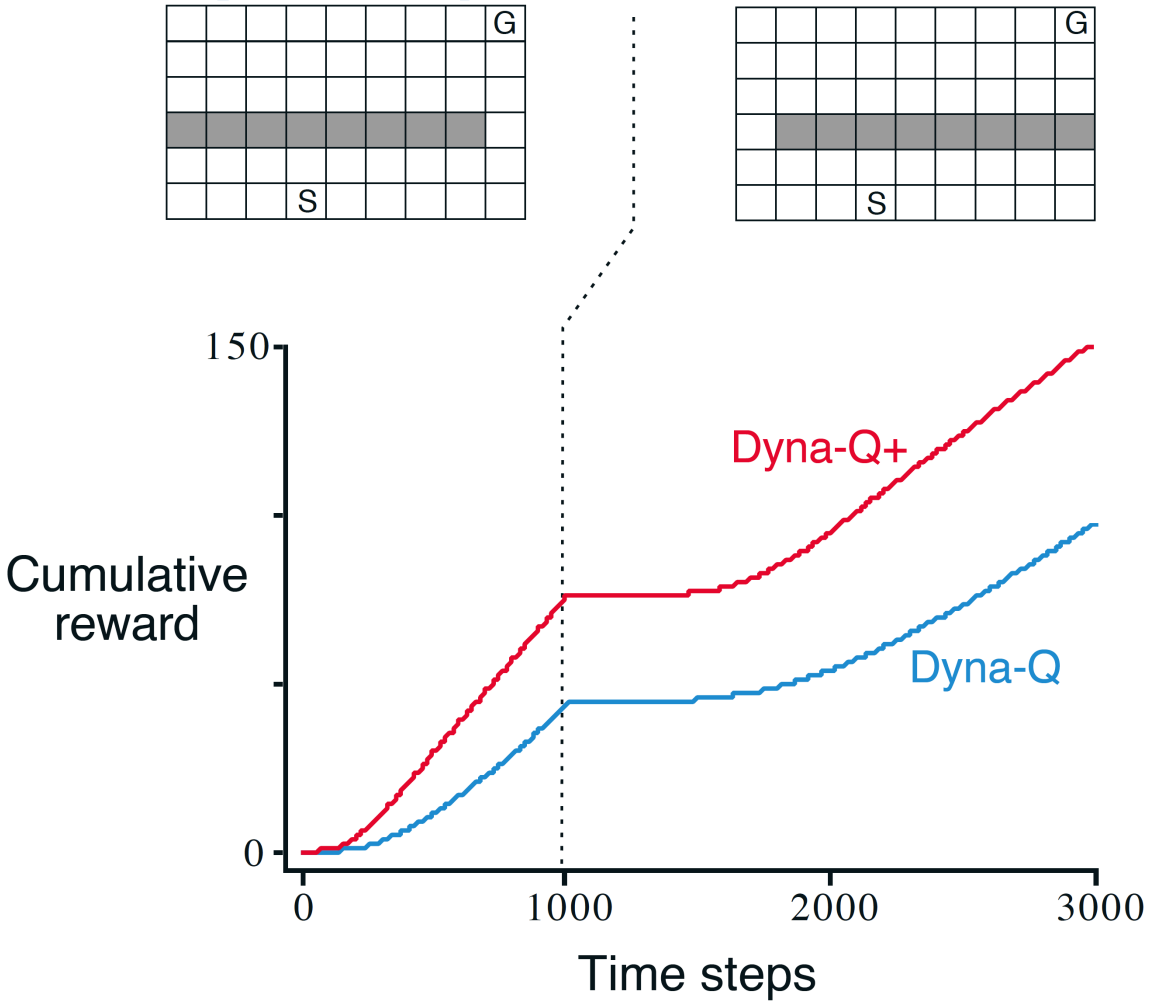
$s \leftarrow$ random previous state

$a \leftarrow$ random previous action

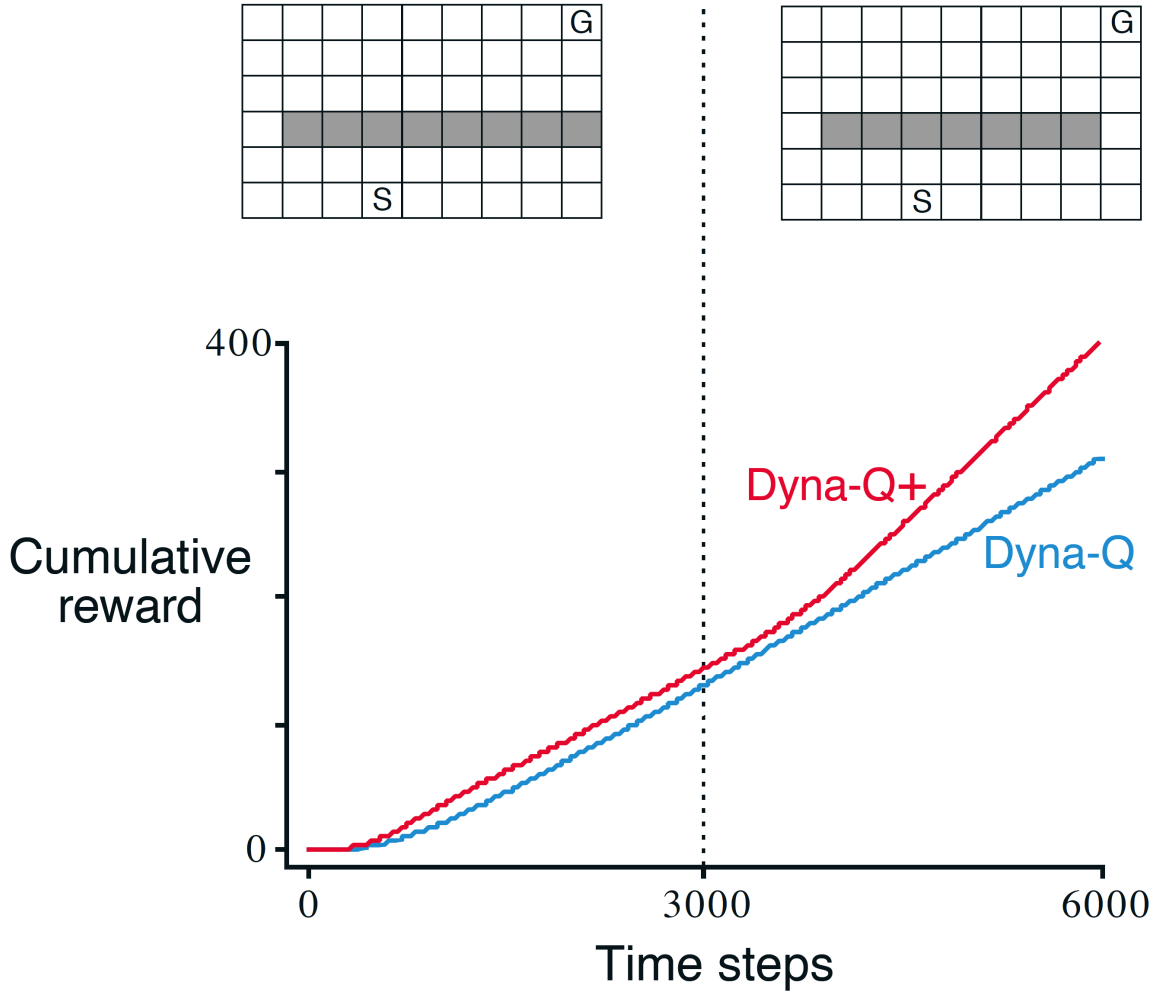
$\hat{r}, s' \leftarrow \textbf{model}(s,a)$

$Q(s,a) \leftarrow Q(s,a) + \alpha\left[\hat{r} + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$

Example: Dyna Maze (2)



Average performance of Dyna agents on a blocking task. The left environment was used for the first **1000** *steps, the right environment for the rest. Dyna-*$Q+$ *is Dyna-*Q *with an exploration bonus that encourages exploration (Source: (Sutton and Barto 1998, Fig. 8.4)).*



Average performance of Dyna agents on a shortcut task. The left environment was used for the first **3000** *steps, the right environment for the rest (Source: (Sutton and Barto 1998, Fig. 8.5)).*

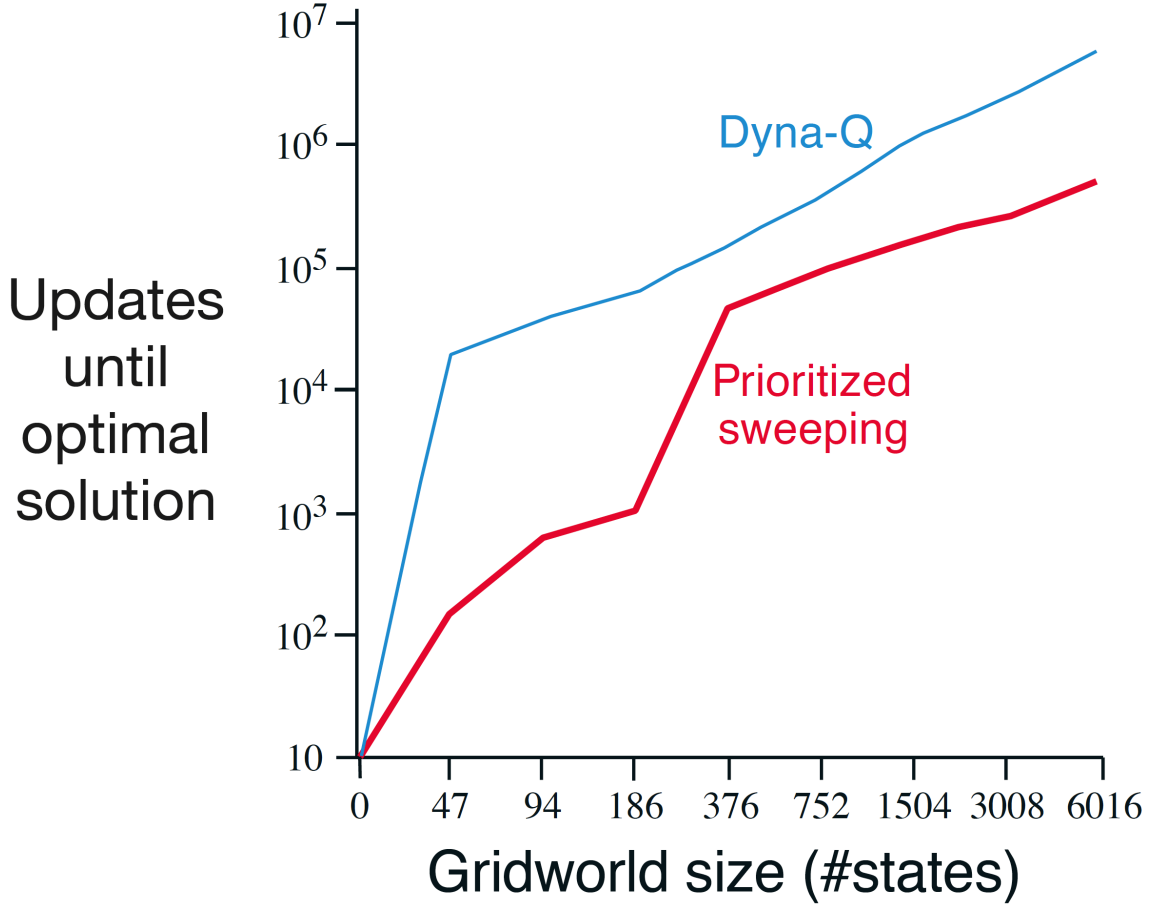
Prioritized sweeping

Algorithm: Prioritized sweeping
initialize: $Q(s,a)$, **model**(s,a) for all s, a,
empty queue $\mathcal{Q}$, threshold $\xi > 0$.

loop forever:
  $a \sim \pi(\cdot|s, Q)$ (e.g. ϵ-greedy)
  Take action a, observe reward r and next state s'
  Add transition to our **model**
  $P \leftarrow \left[r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$
  if $P > \xi$ **then** insert (s,a) into $\mathcal{Q}$ with priority P
  loop: Repeat n times while $\mathcal{Q}$ is not empty
    $(s,a) = \arg\max_P \mathcal{Q}$
    $r, s' \leftarrow$ **model**(s,a)
    $Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{\hat{a}} Q(s', \hat{a}) - Q(s,a)\right]$
    for $\forall(\bar{s}, \bar{a})$ predicted to lead to s:
      $\bar{r} \leftarrow$ predicted reward for $(\bar{s}, \bar{a}, s)$
      $P \leftarrow \left[\bar{r} + \gamma \max_{\hat{a}} Q(s, \hat{a}) - Q(\bar{s}, \bar{a})\right]$
      if $P > \xi$ **then** insert $(\bar{s}, \bar{a})$ into $\mathcal{Q}$ with priority P

- Dyna-Q (randomly) samples from the memory buffer.
  - Many planning updates maybe pointless, e.g., zero-valued state updates during early training.
  - In large state-action spaces: inefficient search since transitions are chosen far away from optimal policies.
- Better: focus on important updates.
  - In episodic tasks: **backward focusing** starting from the goal state.
  - In continuing tasks: **prioritize** according to impact on value updates.
- Solution method is called **prioritized sweeping**.
  - Build up a queue of every state-action pair whose value estimate would change significantly.
  - Prioritize updates by the size of change.
  - Neglect state-action pairs with only minor impact.

Example: Dyna Maze (3)



Prioritized sweeping can dramatically increase the speed at which optimal solutions are found in maze tasks such as this one. These data are for a sequence of maze tasks of exactly the same structure as the first one where we studied varying n, except that they vary in the grid resolution. Both systems made at most $n = 5$ updates per environmental interaction (Source: (Sutton and Barto 1998, Example 8.4)).

Looking into the future: Planning at decision time

Planning at decision time

Background planning (what we have discussed so far)

- Gradually improves policy or value function if time is available.
- Backward view: re-apply gathered experience.
- Feasible for fast execution: policy or value estimate are available with low latency (important, e.g., for real-time control).

Alternative use of a model: Planning at decision time

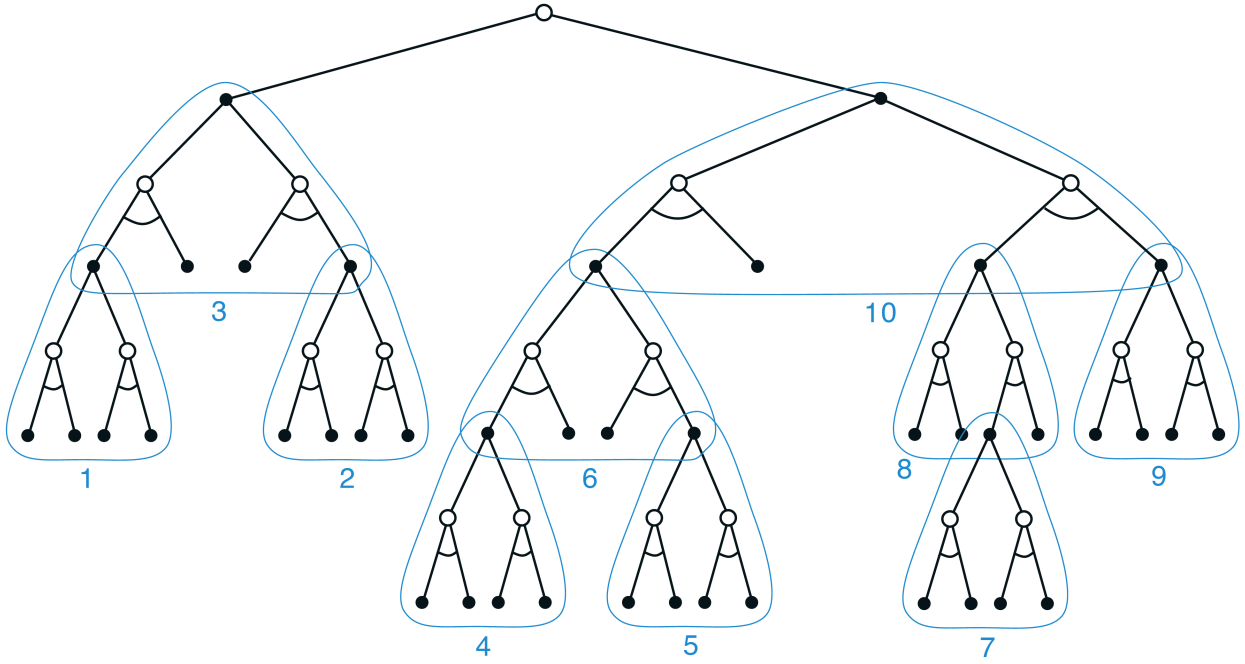
- Select single next future action through planning.
- Forward view: predict future trajectories starting from current state.
- Typically discards previous planning outcomes (start from scratch after state transition).
- If multiple trajectories are independent: easy parallel implementation.
- Most useful if fast responses are not required (e.g., turn-based games or slow systems).

Question: Have we already seen planning algorithms?

- Yes! Optimal control and model predictive control (MPC) fall into this category.
- Now: How to use planning and learning at the same time.
- But first: planning in discrete settings.

Heuristic search

- If we have a model, we can explore all possible state-action sequences from our current state s.
- Challenge: the *curse of dimensionality*. With an increasing prediction horizon, the number of options increases exponentially!
  - For example, let's assume that we have $|\mathcal{A}| = 3$ choices in each step.
  - First prediction step: 3 choices.
  - Second prediction step: 9 choices.
  - Tenth prediction steps: $3^{10} = 59,049$ choices.



Similar to dynamic programming, Heuristic search can be implemented as a sequence of one-step updates (shown here outlined in blue) backing up values from the leaf nodes toward the root. The ordering shown here is for a selective depth-first search (Source: (Sutton and Barto 1998, Fig. 8.9)).

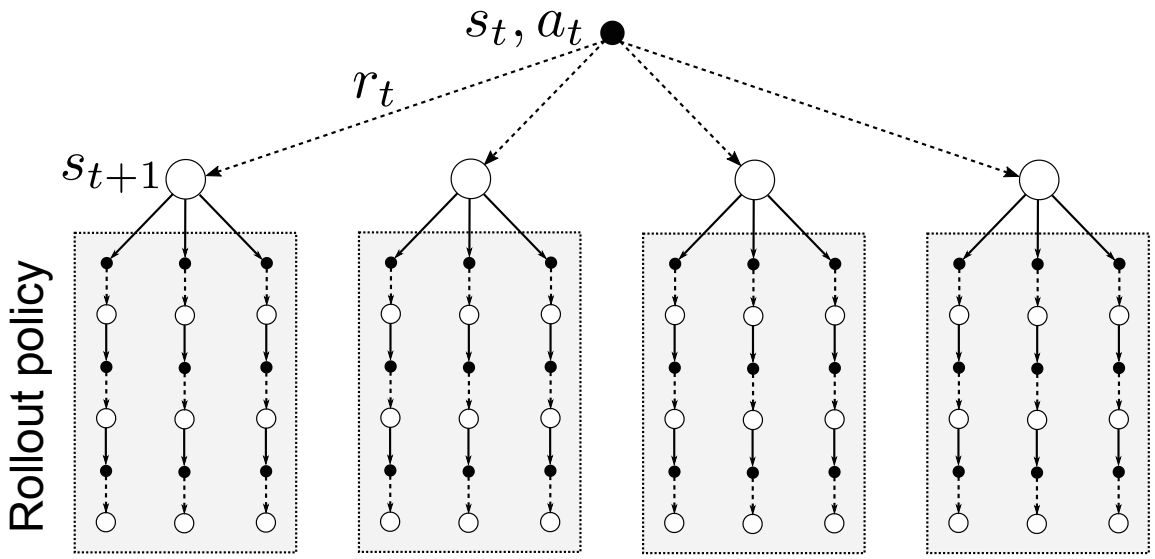
Heuristic search:

- Stop exploration of the tree at some point and replace neglected sequence by some hand crafted heuristic.
- The heuristic has to be very fast such that we can play assess many states and go deep into the tree.
- Drawback: Inaccurate if the heuristic is bad.
  - It can be very hard to find a good heuristic.

Rollout algorithms

Alternative to a heuristic?

- If finding a heuristic is hard, we can instead use a so-called **rollout policy** π^r.
- π^r the doesn't have to be very good, but fast to evaluate.
- Approach: take all possible actions a in s, then for each, follow some rollout policy π^r to the end many times.
- Theoretical argument: policy improvement theorem.
  - If π^r is the same for all experiments and the value for one action is higher than for another, then the policy is better according to the PIT.



Simplified processing diagram of rollout algorithms (Adapted from (Abdelwanis et al. 2026, Fig. 7.12)).

Question: Which function does the rollout serve?

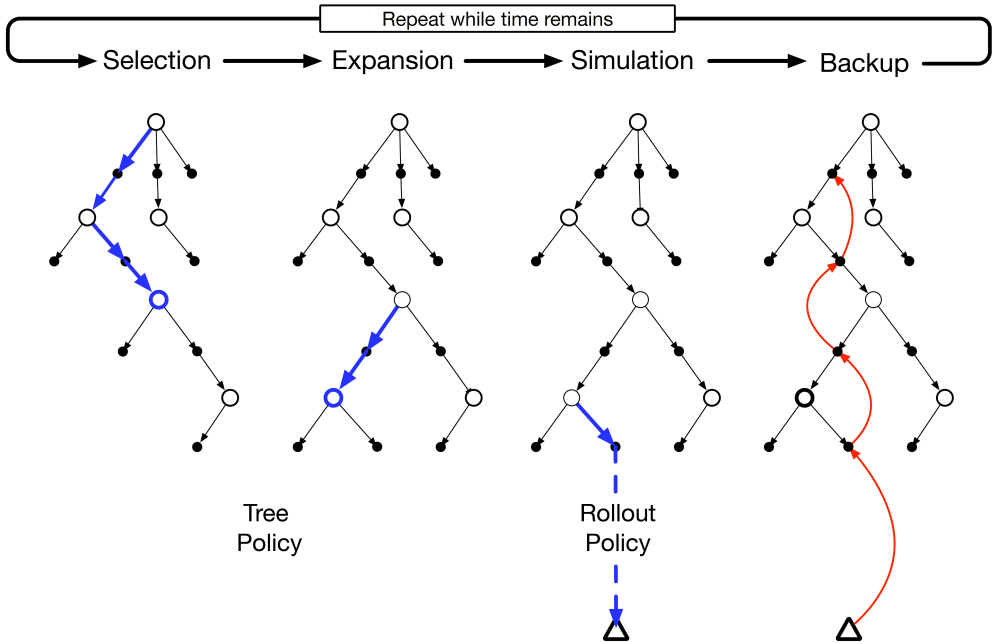
- We have seen a very similar structure in temporal difference learning: The TD target is $y_t = r_t + \gamma V(s_{t+1})$.
- The rollout yields a Monte-Carlo estimate of the value $V(s_{t+1})$ along the branches of the decision tree.

Monte-Carlo Tree Search (MCTS)

- Let's explore MCTS in the special setting of playing games: $r = 1$ if a game is won; $r = 0$ otherwise.
  - We can rate an action by the winning chance (number of wins divided by number of games played with this starting action).

Monte-Carlo Tree Search (MCTS)

1. **Selection** Starting at the root s, navigate down the existing tree by choosing the *most promising* nodes.
   - *Upper Confidence Bound* to balance exploration/exploitation.
   - Move down the tree until a node isn't fully expanded yet.
2. **Expansion** Unless terminal, create new child node(s).
3. **Simulation** Fast *rollout* until win/loss $\Rightarrow r = 1$ / $r = 0$.
4. **Backpropagation** Pass r back through all nodes visited during selection phase and update node statistics (# wins / # visits).



Monte Carlo Tree Search (Source: (Sutton and Barto 1998, Fig. 8.10)).

Execution

- Many MCTS runs: tree becomes highly asymmetric.
  - Deep exploration of paths that look promising.
- When the *thinking time* is up:
  - Compare immediate children of the root node.
  - Choose the one that was visited the most.
- After opponent move, start MCTS again from the new state.

- Little time on paths that lead to quick losses.
- Make that move in the real world.

Exploration vs. exploitation in MCTS

- MCTS treats every choice as a **multi-armed bandit** problem and uses an **upper confidence bound** formula called UCB1:

$$\text{UCB1} = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}}.$$

- The **exploitation** term ($\frac{w_i}{n_i}$):
  - w_i: number of wins (i.e., $r = 1$) simulated through child node i so far.
  - n_i: number of times child node i has been visited.
  - $\frac{w_i}{n_i}$: win rate (or average reward $\mathbb{E}[r]$) of the node.
  - The higher the win rate, the more attractive this node is.
- The **exploration** term ($\sqrt{\frac{\ln N_i}{n_i}}$):
  - N_i is the total number of times the parent node has been visited.
  - If we rarely visit a node, n_i is very small, which makes the exploration term grow large.
- The exploration constant (c) is a parameter we can tune (in theory, it should be $\sqrt{2}$).
- How the balancing works in practice:
  - Imagine a node/move that looks bad at first (maybe it lost its first 2 simulations) $\Rightarrow$ exploitation score is 0.
  - MCTS will start ignoring it to focus on better moves.
  - As the parent visit count N_i grows while n_i remains small, the exploration term steadily increases.
  - Eventually, the exploration bonus becomes so high that it overrides the 0% win rate.
  - If the move fails again ($r = 0$) $\Rightarrow$ exploration shrinks ($n_i \uparrow$); exploitation stays low.

- If it turns out to be a good move ($r = 1$) $\Rightarrow$ exploration shrinks ($n_i \uparrow$); exploitation goes up $\mathbb{E}[r] \uparrow$.

AlphaGo & AlphaZero

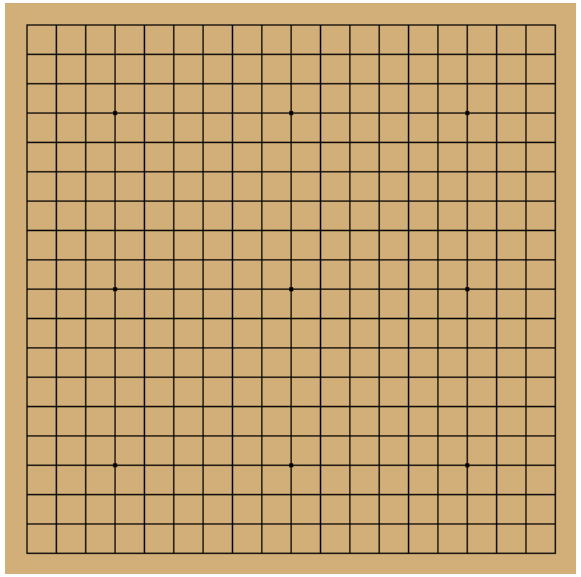
AlphaGo & AlphaZero

The rules of Go (strongly simplified)

- Black and White place stones in turns on a 19×19 grid.
- If a stone (or a group of stones) is surrounded by the opponent: captured/removed.
- Score: number of captured stones plus number of surrounded fields on the board.

Complexity (see David Silver's NeurIPS 2017 presentation)

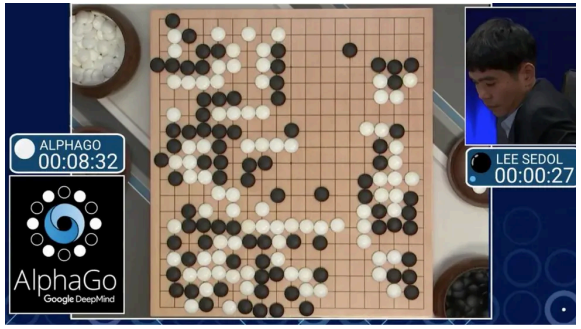
- Estimated number of possible board positions: 10^{170}.
- Approximate number of game sequences: 200^{200}.
- **Conclusion**: Completely untractable for classical search methods!



The first 150 moves of a Go game (Source).

AlphaGo & AlphaZero

- AlphaGo was the first program to beat a professional player as well as a world champion.
  - Trained on a large basis of positions assessed by human experts.
  - Then fine-tuned via self-play.
- AlphaZero: Strongly simplified architecture, trained exclusively through self-play.



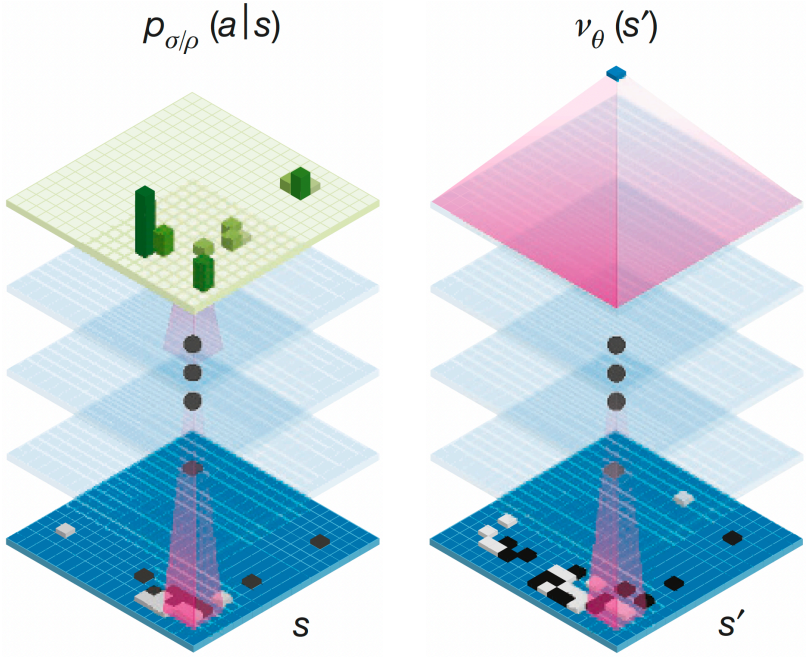
AlphaGo (Silver et al. 2016) (Source).

AlphaGo – Architecture

The AlphaGo architecture consists of several networks

1. Expert-trained policy network $p_\sigma(a|s)$.
   - supervised training on 30 million moves (from the KGS Go Server).
   - Architecture:
     - $19 \times 19 \times 48$ input features,
     - 11 hidden layers with 192 channels, 3×3 conv. kernels, ReLU activation,
     - 1×1 convolution filter with softmax activation,

   $\Rightarrow$ probability distribution over $19 \times 19 + 1$ choices (including *pass*).
2. Improved policy network $p_\rho(a|s)$ by self-play.
   - Same architecture as supervised learning policy.
3. Value network $v_\theta(s')$
   - Predicts a scalar value $V \in [-1, 1]$ for the probability of white winning from this position:
     - $V = -1$: certain loss,
     - $V = +1$: certain victory.
   - Same architecture as the policies, except for different readout.
4. Linear policy network p_π for very fast rollouts. ($\approx 2\mu s$ vs. $\approx 3ms$).



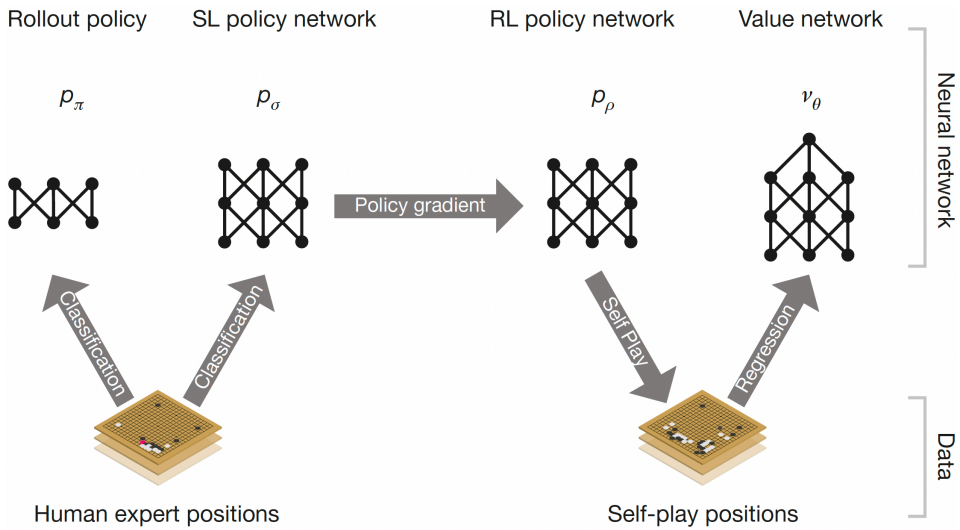
Policy networks Value network

Source: (Silver et al. 2016).

AlphaGo – Training

Training procedure

1. Train **SL policy** $p_\sigma(a|s)$ $\Rightarrow$ supervised with cross-entropy loss.
   - given an input-output pair (s, a) from the dataset, make a one-hot encoding of a and minimize the cross-entropy loss.
   - $p_\sigma(a|s)$ achieved an accuracy of 57% for predicting players' moves.
2. Train the **rollout policy** $p_\pi(a|s)$ in the same way as the SL policy.
   - $p_\pi(a|s)$ achieved an accuracy of $\approx 24\%$ for predicting players' moves.
   - For the purpose of rollouts, this is a reasonable performance.
   - The more important factor is the very fast inference.



3. Train the **RL policy** $p_\rho(a|s)$ via REINFORCE (sample $\rightarrow$ policy gradient $\rightarrow$ gradient ascent) with initial guess $p_\sigma(a|s)$.
   - Monte-Carlo sampling over entire game trajectories.
   - High-variance problem $\Rightarrow$ very large number of simulated games.
4. Train the **value network** $v_\theta(s')$ in a supervised fashion via Monte-Carlo sampling.
   - Use the RL policy $p_\rho(a|s)$ to create a large dataset (30 million samples) of near-optimal games.

AlphaGo – Execution/Play

The execution phase of AlphaGo is a version of MCTS

1. **Selection**: As in MCTS, but using *polynomial upper confidence trees*:

$$a_t = \arg\max_a \left(Q(s,a) + U(s,a)\right)$$

2. **Expansion**: Generate the legal moves from the leaf position.
3. **Simulation**: Dual evaluation system:
   - The value network $v_\theta(s')$ estimates the chance of winning.
   - The rollout policy $p_\pi(a|s)$ is used to play many games to the end.
   - The assessed value is a mixture of the two (often 50/50).
4. **Backup**: Update node visitation counts and winning percentages.

Selecting the actual move

- AlphaGo does **not** necessarily pick the move with the highest value.
- It plays the move that was visited the most times during simulations.
- This ensures that the chosen move is stable and has been thoroughly evaluated through multiple simulation paths.

Polynomial upper confidence trees (PUCT)

- **Exploitation term $Q(s,a)$**: averaging the values of the leave states s_L via $v_\theta(s_L)$ over the plays during the simulation step.
- **Exploration term $U(s,a)$**: a version of UCB,

$$U(s,a) = c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}.$$

  - $N(s,a)$ is the visit count.
  - $\sum_b N(s,b)$ is the total visit count of the parent node.
  - c_{puct} is a hyperparameter.
  - $P(s,a)$ is the prior probability provided by the SL Policy Network $p_\sigma(a|s)$.

AlphaGo – Exploitation-exploration mechanics

When AlphaGo looks at a new board state during a simulation, this is what happens dynamically:

1. **Setting the prior**: The SL policy network $p_\sigma(a|s)$ outputs a probability distribution $P(s,a)$ for all legal moves.
   - "Expert" moves that look professional / human: large P (e.g., 0.60).
   - Odd or sub-optimal moves: small P (e.g., 0.001).
   - This $P(s,a)$ stays fixed for the rest of the search phase; it acts as a permanent multiplier for the exploration bonus.

$$a_t = \arg\max_a \left(Q(s,a) + U(s,a)\right)$$

$$U(s,a) = c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

2. **Early simulations (policy dominates)**:
   - At the start of the search, the visit counts $N(s,a)$ for all moves are 0.
   - Because Q is uninitialized or 0, selection is dominated by exploration $U(s,a) \Rightarrow$ driven by the policy network's $P(s,a)$.
   - As a result, AlphaGo's first few simulations will only explore the top moves suggested by the human SL policy.
3. **Deep search (UCB dominates)**:
   - As a specific move a is frequently selected, its visit count $N(s,a)$ grows.
   - Exploration bonus $U(s,a)$ decays $\Rightarrow$ selection relies more on the actual win rate $Q(s,a)$ discovered by the simulations.
   - Conversely, if a move has a decent prior probability $P(s,a)$ but AlphaGo has ignored it for a while, the numerator $\sqrt{\sum_b N(s,b)}$ keeps growing while its own $N(s,a)$ stays stagnant. This causes its exploration bonus to grow.

AlphaZero

The AlphaZero framework (Silver et al. 2017) beat AlphaGo 100-0 (!) **and** can play multiple games (Go, Chess, Shogi).

The main changes

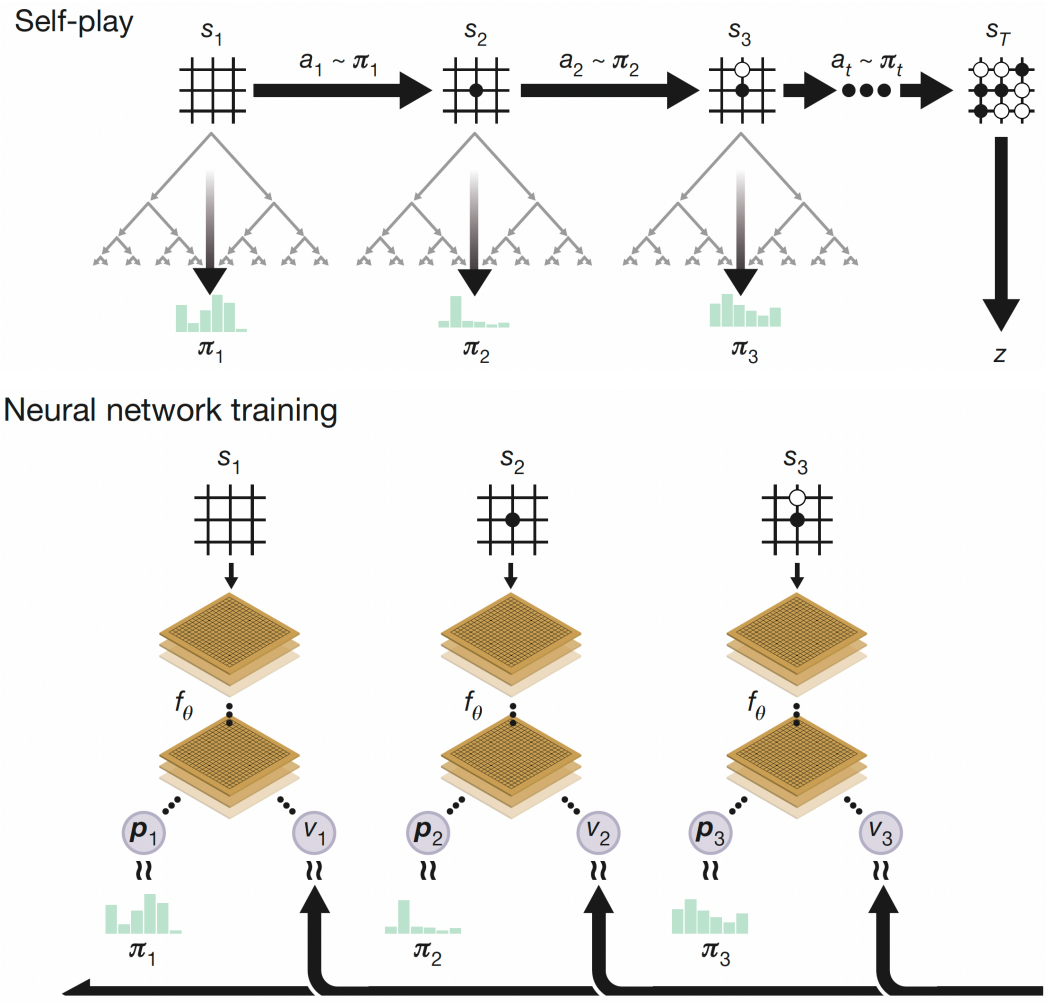
1. **Simpler architecture**. Single "two-headed" network:
   - Input: Raw board position, no features.
   - Output: $(p, v) = f_\theta \Rightarrow$ move probabilities (i.e., policy) and winning chance (i.e., value).
   - Training:

$$\min_\theta (z - v)^2 - \pi^\top \log p + c\|\theta\|^2 \qquad (1)$$

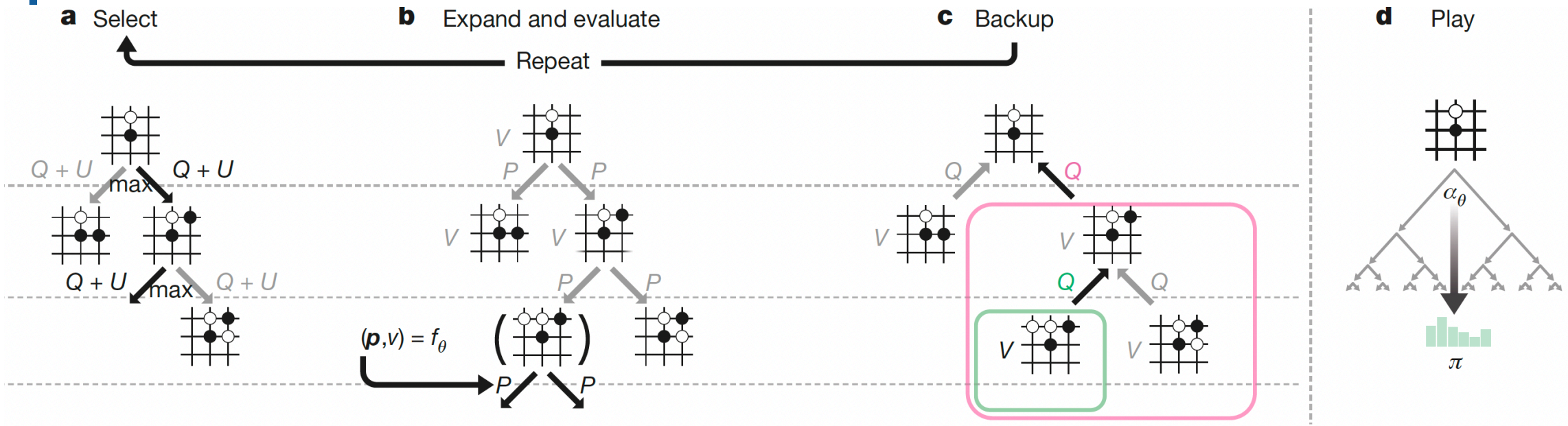
   - Value v matches game outcome $z \in \{-1, 0, +1\}$.
   - Policy network p matches MCTS probabilities π.

2. **No human data**. Starts with random weights:
   - Learns entirely via RL self-play from game zero.

3. **Pure MCTS without rollouts**.
   - The evaluation of a leaf node comes solely from the value head of the single neural network.
   - MCTS is strictly used as a policy improvement operator.



AlphaZero – MCTS



Over the course of training,

- 4.9 million games of self-play were generated,
- using 1,600 simulations for each MCTS,
- which corresponds to approximately 0.4 s thinking time per move.

💡 α_θ represents the final policy (f_θ plus MCTS) after training is complete.

AlphaZero – Training procedure

1. **MCTS**: From the current actual board state s, run a few thousand MCTSs.

- The neural network's policy head provides prior probabilities to guide which branches of the tree to explore.
- The neural network's value head evaluates the leaf nodes without needing random rollouts.

2. **Extracting the search probabilities (π)**

- After the MCTS simulations are complete, the choices are aggregated.
- Counts state visitations and convert into probability vector π.
  - π represents a much stronger policy than the raw output of the neural network.

3. **Choosing the move** based on π.

- For the first several moves of a training game, select moves probabilistically proportional to π to ensure deep exploration of the state space.
- For the rest of the game, greedily selects the most visited move.

4. **The game outcome**: $z = +1$ for a win, $z = 0$ for a draw and $z = -1$ for a loss.

Generating the training data for f_θ

- Every state s encountered during a self-play game becomes a data point for the training buffer (e.g., if a game lasted 60 moves, it yields 60 distinct training samples).
- Each training sample is a triplet (s, π, z)
  - s: The board state (input).
  - π: The search probabilities calculated by MCTS for that state (target for the policy head).
  - z: The winner of that specific game (target for the value head).
- Supervised learning of f_θ via (1)!

Learning a model

Surrogate modeling / world models

If we do not have a model, but still want to use model-based RL, we can try to **learn a model from experience**.

Procedure:

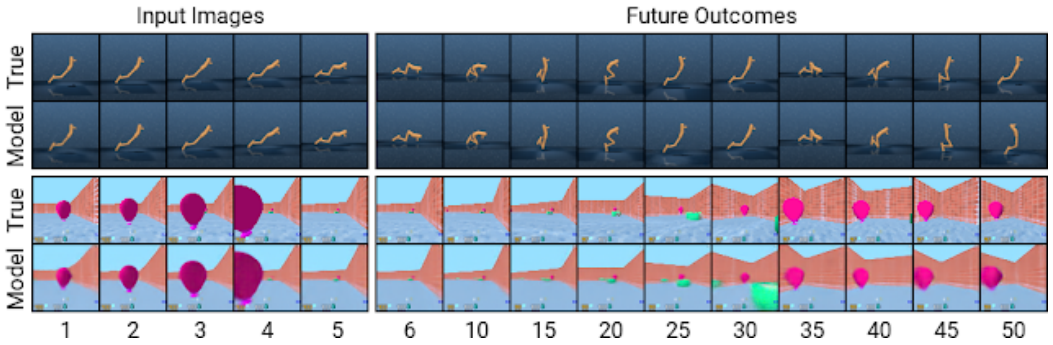
- Collect **sample sequences** follwing some policy π.
- Optional: Learn a compression into some **latent representation**.
- **Train predictor** for new (latent) states and rewards.
- Optional: **Decode** latent states to full states (or observations).
- We can then perform RL on this **world model**.

Challenges & questions

- Data acquisition
  - Which data should we use?
  - How much data?
  - Collected under which actions?
- The policy changes the state distribution: ρ_π.
- Intertwining RL and model learning?



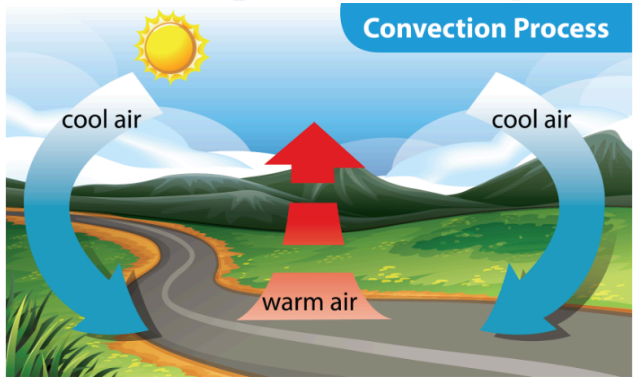
Wolrd model (Source: Danijar Hafner's Google Research blog post).

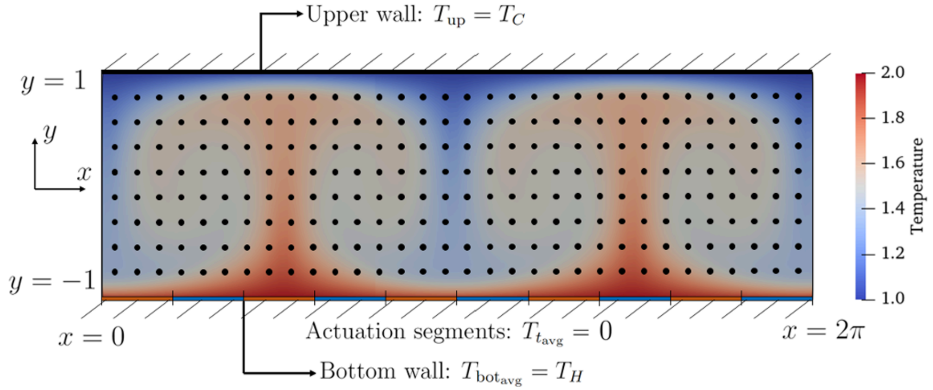


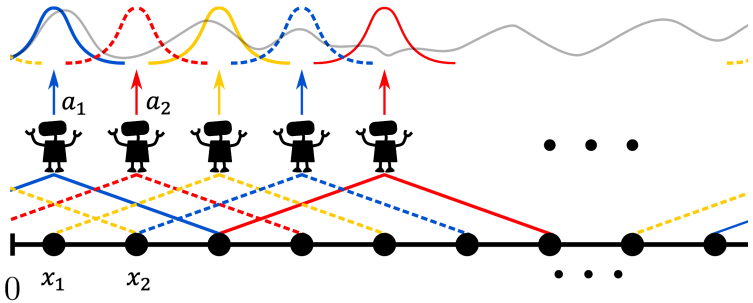
Prediction (Source: Danijar Hafner's Google Research blog post).

💡 In the mathematics / dynamical systems communities, world models have been studied for a very long time under the term **surrogate models**.

Example: Rayleigh-Bénard convection (1)

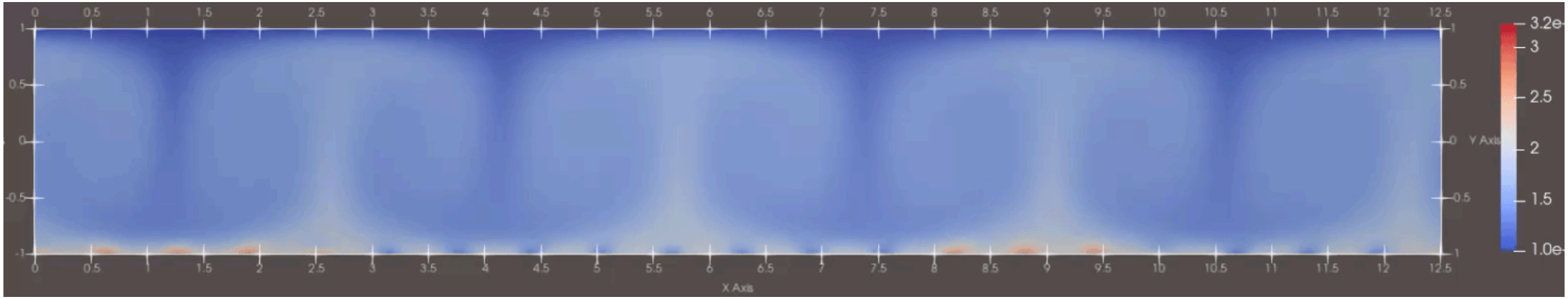






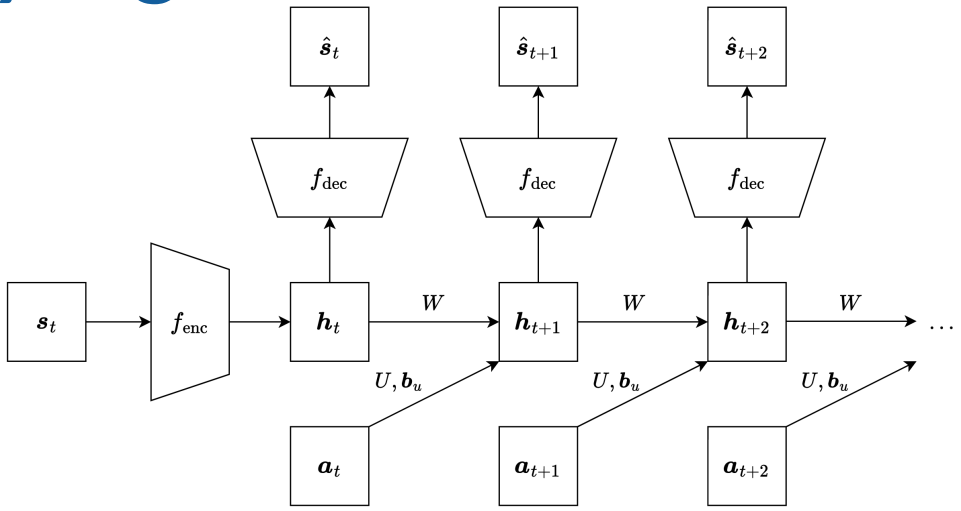
Making use of symmetries, we can replace a single large agent by many identical, sm
(Source: (Peitz et al. 2024)).

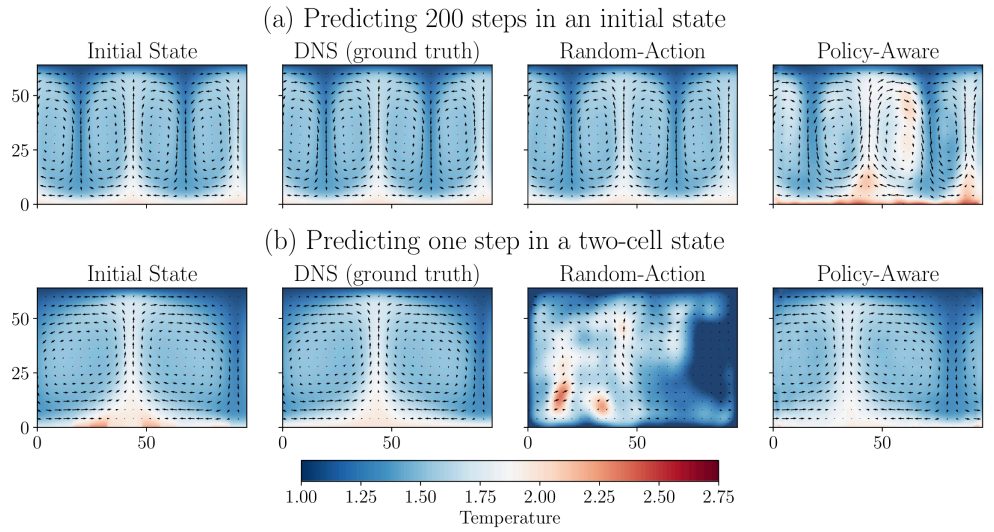
Multi-agent RL → merging of convection cells reduces the convective heat transport

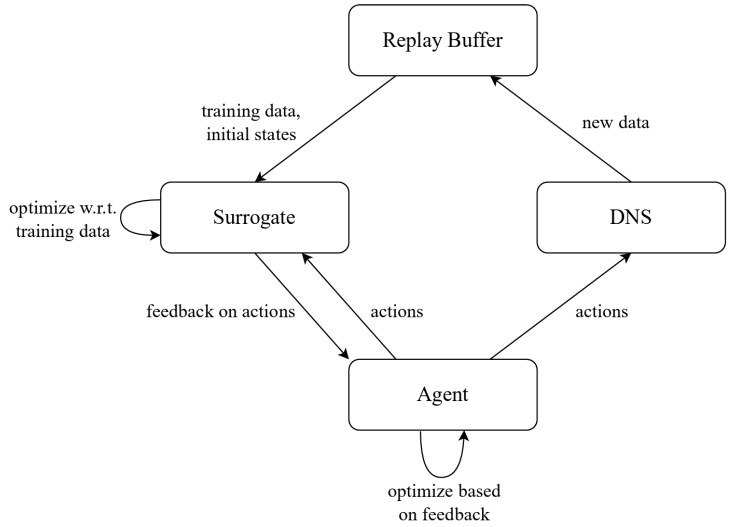


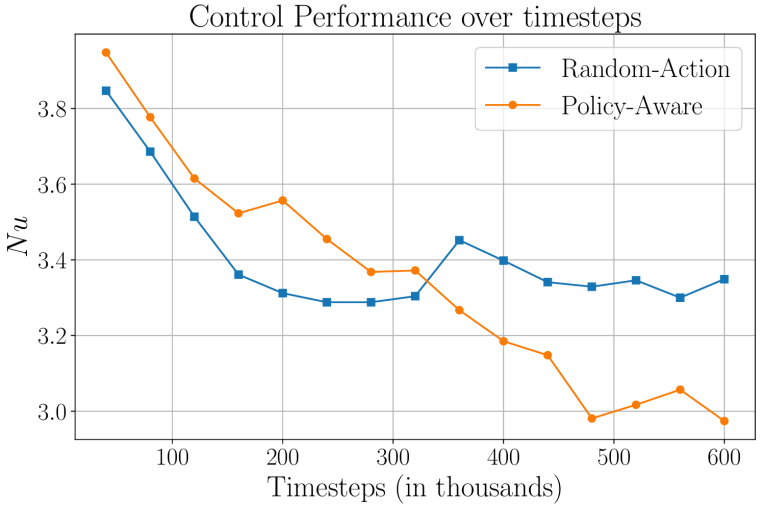
Example: Rayleigh-Bénard convection (2)

- World model: Autoencoder plus linear model in latent space.
- Intertwined world modeling and policy learning.
- Strictly required for ensuring model performance.
  - Otherwise, prediction before / after cell merging is impossible to predict (see bottom left).
  - Policy-aware data collection strongly improves RL









Results from (Plotzki and Peitz 2026)

performance
(bottom right).

The dreamer architecture

- Do we have to use some standard algorithm such as PPO or SAC with a learned world model?
- Such models are usually end-to-end differentiable via backpropagation.
  $\Rightarrow$ Differentiable Predictive Control (DPC)!
- That's the concept of the **Dreamer** architectures (Hafner et al. 2020; Hafner et al. 2021; Hafner et al. 2025):
  - Learn a world model.
  - Model rollout over a small number of steps (e.g., $p = 15$) following the current policy π_ϕ.
  - Backpropagation through time (BPTT) to determine the gradient of the policy parameters ϕ w.r.t. the closed-loop performance measure.
  - Execute policy on real environment and collect new experience.
  - Repeat.



o_1

Source: *Danijar Hafner's Google Research blog post.*



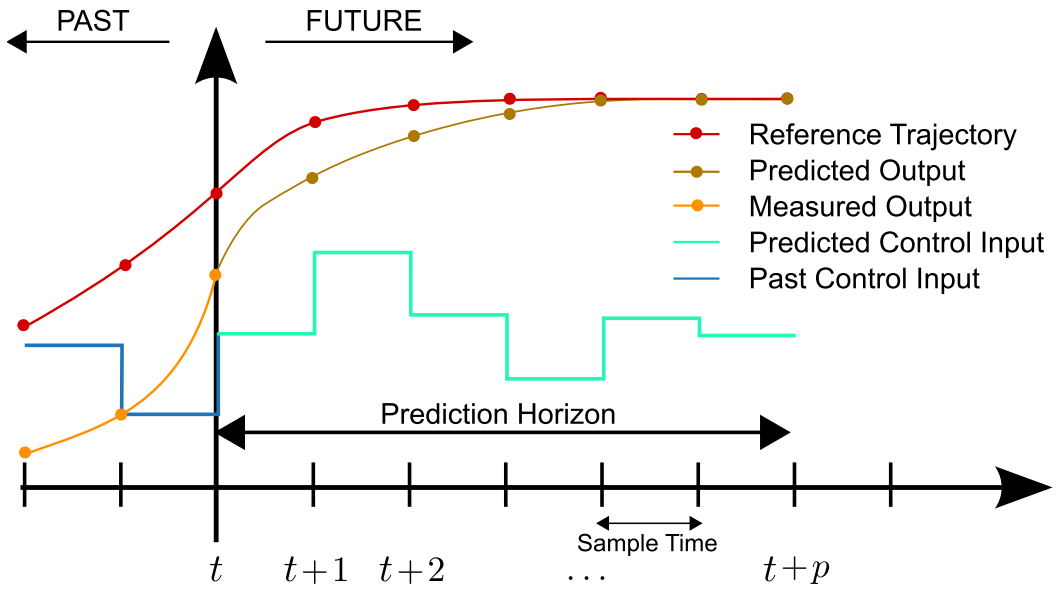
Source: *Danijar Hafner's Google Research blog post.*

Data-driven MPC

Similar to DPC, we can also use world models in the MPC context:

- Learn a dynamics model from sampled sequences.
- Use the model in an online open-loop optimal control problem.
- Close the loop via MPC feedback (initialize the OCP with the measured state $\boldsymbol{s_t}$).

Advantage: Real-time capability if the world model is fast



Source: Wikipedia.

Summary / what we have learned

- Revisiting the past: Dyna-style algorithms
  - Dyna-Q and Dyna-$Q+$ revisit past state transitions
  - With newer Q approximations, we can learn from the same experience multiple times
  - Prioritized sweeping allows us to revisit more important updates more frequently
- Looking into the future: Planning at decision time using predictive models
  - Heuristic search, rollouts and MCTS to obtain MC estimates of the value function at a given state
- AlphaGo & AlphaZero
- Learning a model
  - World models / surrogate models from sampled trajectories
  - Interplay between modeling and RL is very important
  - Dreamer architecture = DPC with world models

References

Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. “Reinforcement Learning: A Comprehensive Open-Source Course.” *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:10.21105/jose.00306.

Hafner, Danijar, Timothy P Lillicrap, Mohammad Norouzi, and Jimmy Ba. 2021. “Mastering Atari with Discrete World Models.” In *International Conference on Learning Representations (ICLR)*.

Hafner, Danijar, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. 2020. “Dream to Control: Learning Behaviors by Latent Imagination.” In *International Conference on Learning Representations (ICLR)*.

Hafner, Danijar, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 2025. “Mastering Diverse Control Tasks Through World Models.” *Nature* 640 (8059): 647–53.

Peitz, Sebastian, Jan Stenner, Vikas Chidananda, Oliver Wallscheid, Steven L. Brunton, and Kunihiko Taira. 2024. “Distributed Control of Partial Differential Equations Using Convolutional Reinforcement Learning.” *Physica D: Nonlinear Phenomena* 461: 134096.